Day 126: Autoencoders From Scratch: a Real Reconstruction Demonstration

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 1 | Complete study guide

Watch: Autoencoders - Explained (DataMListic) | Autoencoder Explained - Deep Neural Networks (AIEngineering)

Script: `day126_autoencoders.py` · every number in this guide is copied by program from `day126_run_output.txt`, the real output of that script.

Week 19 begins: networks that learn from images alone, with no labels. An autoencoder squeezes each image through a few numbers and rebuilds it. Written from scratch and trained on real MNIST digits, a linear one lands on PCA's answer, as theory says it must (0.01707 against 0.01689); with ReLU layers it rebuilds far better than PCA (0.00731 against 0.01689 with 32 numbers). The codes it learns are no better than PCA's for finding similar digits (0.9155 against 0.9170), and codes it never saw decode to blurs: the gaps that Day 129's variational autoencoder is built to fill.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

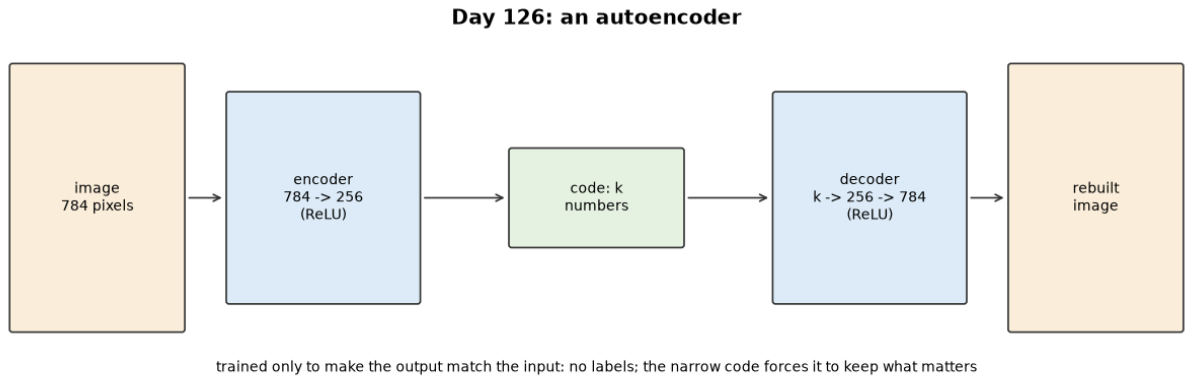
An **autoencoder** is a network trained to output its own input. The first half, the **encoder**, maps the image to a short **code**; the second half, the **decoder**, rebuilds the image from the code. Because the code is narrow, the network cannot just copy: it must find the few numbers that describe most images.

Nothing but the images themselves is used to train it. That makes it **unsupervised**, or more precisely **self-supervised**: the input is its own target.

REAL-WORLD SCENARIO

Describing a friend's face over the phone in twenty words, so that a sketch artist can draw it. The words that matter (round face, glasses, short hair) are the code; the artist is the decoder.

SYSTEM DIAGRAM



Encoder, code, decoder: trained only to make the output match the input.

Today's part for the week's build

`day126_exercise.py` is the network's machinery, checked by hand: gradients, a line rediscovered, a loss that falls. Its real output:

ACTUAL OUTPUT

```
gradient check, bce: largest gap 1.5e-09
gradient check, mse: largest gap 2.2e-09
points on y = 2x: decoder direction (0.4472, 0.8944); by hand (1, 2)/sqrt(5) = (0.4472, 0.8944);
    error 4.34e-12
6-4-6 network, cross-entropy per image: 4.158 after epoch 1, 0.316 after 60; falls every 10
    epochs: True
```

Data: MNIST, 70,000 handwritten digits of 28 x 28 pixels, fetched once from openml.org through scikit-learn and cached outside the repository; the first 20,000 training images and the official 10,000 test images. Labels are used only to judge. Day 123's PCA and Day 119's k-means are imported.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: a 2-1-2 autoencoder

DEFINITION — start here, no prior knowledge assumed

41 points near the line y = 2x, squeezed through one number. A linear network with a one-number code can only keep each point's position along one direction, so it should find the line.

REAL-WORLD SCENARIO

Describing where houses stand on a straight road by one number each: the distance from the road's start.

MATHEMATICS

$$code\ c = W_1 x + b_1; \quad rebuilt\ x' = W_2 c + b_2; \quad loss = |x' - x|^2$$

IMPLEMENTATION

```
def init_net(sizes, seed):
    rng = np.random.RandomState(seed)
    return [((rng.randn(a, b) * np.sqrt(2.0 / a)).astype(np.float32), np.zeros(b, np.float32)) for
        a, b in zip(sizes[:-1], sizes[1:])]
def forward(net, X, acts):
    A = [X]
    for (W, b), act in zip(net, acts):
        Z = A[-1] @ W + b
        A.append(np.maximum(Z, 0) if act == "relu" else Z)
    return A
def backward(net, A, acts, dout):
    grads = [None] * len(net)
    for i in range(len(net) - 1, -1, -1):
        dZ = dout * (A[i + 1] > 0) if acts[i] == "relu" else dout
        grads[i] = (A[i].T @ dZ, dZ.sum(0))
        dout = dZ @ net[i][0].T
    return grads, dout
```

ACTUAL OUTPUT

```
squared error per point: 13.8091 after the first epoch, 0.0128 after 2,000
the decoder's direction (0.4367, 0.8996); PCA's first component of the same noisy points (0.4352,
    0.9003); the line without noise (1, 2)/sqrt(5) = (0.4472, 0.8944)
one number per point (its position along the line) is all the code can hold, so the network learns
    the line
Actual output from this lesson's run.
```

What it means

The error falls from 13.8091 to 0.0128, and the decoder's direction (0.4367, 0.8996) matches PCA's first component of the same noisy points (0.4352, 0.9003): both find the best line through the noise, close to the clean line's (0.4472, 0.8944).

WHY CHOOSE THIS?

A case small enough to know the answer.

WHY NOT THIS? (TRADEOFFS)

Linear and one-dimensional: real images need more (section 4).

2. Correctness: a gradient check

DEFINITION — start here, no prior knowledge assumed

A small network (6-5-3-6, ReLU), every weight and bias nudged both ways, for both losses.

REAL-WORLD SCENARIO

Checking a car's speedometer against the time it takes to pass two mileposts.

MATHEMATICS

$$\textit{cross-entropy: } L = \Sigma\,[\log(1 + e^{l}) - x\,l],\ dL/dl = \text{sigmoid}(l) - x; \quad \textit{squared error: } L = \Sigma(l - x)^2,\ dL/dl = 2(l - x)$$

IMPLEMENTATION

```
def recon_loss(L, X, kind):
    n = len(X)
    if kind == "bce":
        return float((np.logaddexp(0, L) - X * L).sum() / n), (sigmoid(L) - X) / n
    return float(((L - X) ** 2).sum() / n), 2 * (L - X) / n
```

ACTUAL OUTPUT

```
  bce: largest gap over all 77 parameters 1.5e-09
  mse: largest gap over all 77 parameters 2.2e-09
Actual output from this lesson's run.
```

What it means

Backpropagation agrees with finite differences to 1.5e-09 and 2.2e-09 over all 77 parameters. Cross-entropy treats each pixel as a probability of ink, which suits MNIST's mostly black-or-white pixels; it is computed from the logits so that it never takes the log of 0.

WHY CHOOSE THIS?

Every result rests on checked gradients.

WHY NOT THIS? (TRADEOFFS)

A correct gradient does not make training succeed.

3. A linear autoencoder rediscovers PCA

DEFINITION — start here, no prior knowledge assumed

784 -> 32 -> 784 with no ReLU and squared error, trained by Adam for 40 epochs, against PCA with 32 components.

REAL-WORLD SCENARIO

Two routes to the same summary: one computed in a single step, one found by trial and error.

IMPLEMENTATION

```
class Adam:
    def __init__(self, net, lr=1e-3):
        self.lr, self.t = lr, 0
        self.m = [(np.zeros_like(W), np.zeros_like(b)) for W, b in net]; self.v =
            [(np.zeros_like(W), np.zeros_like(b)) for W, b in net]
    def step(self, net, grads):
        self.t += 1; c1, c2 = 1 - 0.9 ** self.t, 1 - 0.999 ** self.t
        for (W, b), (gW, gb), m, v in zip(net, grads, self.m, self.v):
            for p, g, mm, vv in ((W, gW, m[0], v[0]), (b, gb, m[1], v[1])):
                mm *= 0.9; mm += 0.1 * g; vv *= 0.999; vv += 0.001 * g * g
                p -= self.lr * (mm / c1) / (np.sqrt(vv / c2) + 1e-8)
```

ACTUAL OUTPUT

```
test error per pixel: linear autoencoder 0.01707 after 40 epochs (54 s); PCA with 32 components 0.01689
held-out error every 10 epochs: [13.665, 13.394, 13.422, 13.383] (squared error summed per image)
the decoder's subspace against PCA's: cosines of the principal angles from 0.9999 down to 0.9878 (1 = the same
    direction)
a linear autoencoder can only learn PCA's subspace; it may not find all of it, and gradient descent gets there
    slowly
Actual output from this lesson's run.
```

What it means

The linear autoencoder ends at 0.01707 per pixel against PCA's 0.01689, and its 32 directions span almost the same subspace (principal-angle cosines from 0.9999 down to 0.9878). Theory says the best a linear autoencoder can do is PCA's subspace; gradient descent approaches it but, after 40 epochs, has not quite arrived. PCA gets there exactly, in one step.

WHY CHOOSE THIS?

Shows what the nonlinearity adds by taking it away.

WHY NOT THIS? (TRADEOFFS)

As a way of computing PCA it is slower and less exact.

4. Nonlinear against PCA

DEFINITION — start here, no prior knowledge assumed

784 -> 256 -> k -> 256 -> 784 with ReLU and cross-entropy, 20 epochs, for k = 2, 8 and 32, against PCA with k components.

REAL-WORLD SCENARIO

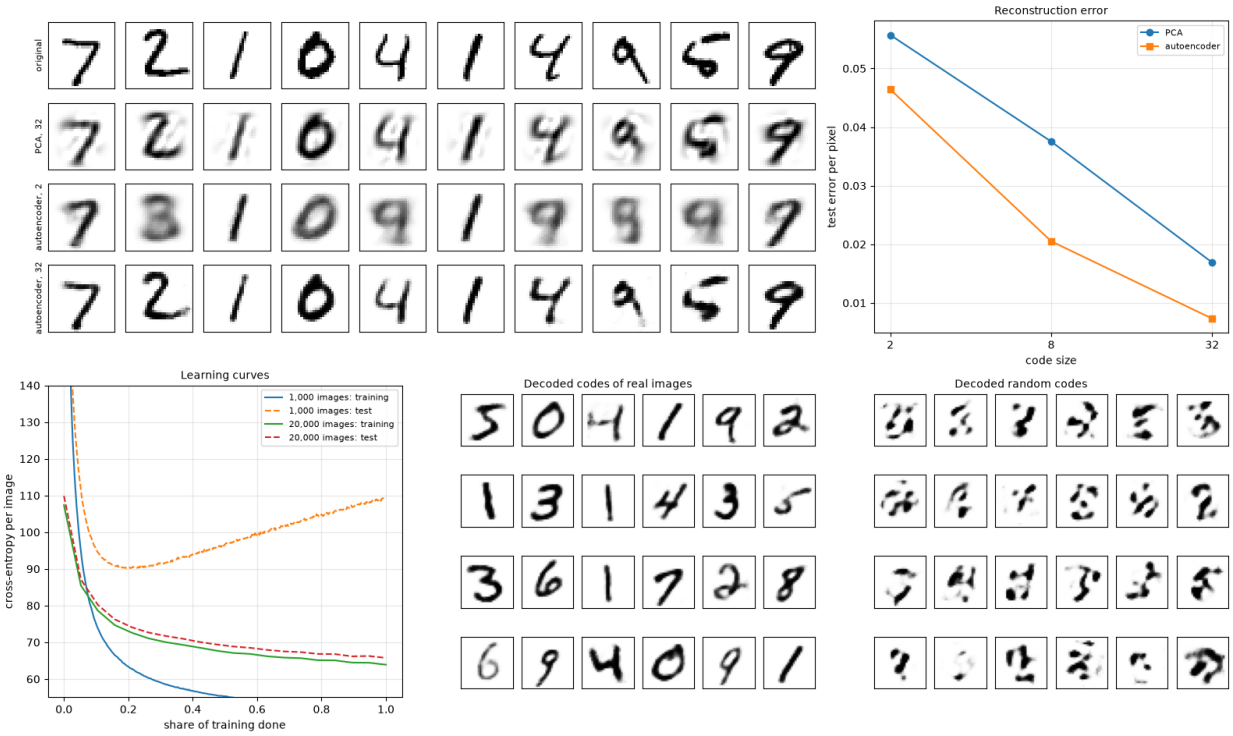
Folding a map along its creases instead of only projecting its shadow.

IMPLEMENTATION

```
def train_ae(X, sizes, acts, kind="bce", epochs=20, batch=128, lr=1e-3, seed=0, Xval=None,
    noise=None):
    net = init_net(sizes, seed); opt = Adam(net, lr); rng = np.random.RandomState(seed + 1); hist
        = []
    for _ in range(epochs):
        o = rng.permutation(len(X))
        for s in range(0, len(X), batch):
            xb = X[o[s:s + batch]]; xin = xb if noise is None else noise(xb, rng)
            A = forward(net, xin, acts); _, d = recon_loss(A[-1], xb, kind)
            opt.step(net, backward(net, A, acts, d)[0])
        hist.append((recon_loss(forward(net, X[:2000], acts)[-1], X[:2000], kind)[0],
                     None if Xval is None else recon_loss(forward(net, Xval, acts)[-1], Xval,
                         kind)[0]))
    return net, hist
```

ACTUAL OUTPUT

```
   code size       PCA  autoencoder  seconds
           2   0.05572      0.04643      102
           8   0.03753      0.02051       90
          32   0.01689      0.00731      109
test error per pixel (pixels from 0 to 1); the same 10,000 test images for both
Actual output from this lesson's run.
```



Top: ten test digits, rebuilt by PCA (32), the autoencoder with 2 numbers and with 32. Top right: error against code size. Bottom: learning curves with 1,000 and 20,000 images; decoded codes of real images; decoded random codes.

What it means

The autoencoder wins at every size, and by more as k grows: with 32 numbers 0.00731 against 0.01689, with 8 0.02051 against 0.03753. With 2 numbers (0.04643) the digits are recognisable but drift towards a few typical shapes (a 2 rebuilt as a 3 in the figure). Each network took about 100 seconds on 20,000 images.

WHY CHOOSE THIS?

Curved compression: much better reconstructions for the same code size.

WHY NOT THIS? (TRADEOFFS)

Training, settings and a random start, where PCA is one exact step.

5. How much data

DEFINITION — start here, no prior knowledge assumed

The same 32-number network, trained on 1,000 images for 400 epochs and on 20,000 for 20, so both see 400,000 images.

REAL-WORLD SCENARIO

Memorising one textbook's answers against learning from many textbooks.

ACTUAL OUTPUT

```
    1000 training images: error per pixel on them 0.00161, on the test images 0.01827; gap
        +0.01666
   20000 training images: error per pixel on them 0.00664, on the test images 0.00731; gap
       +0.00068
Actual output from this lesson's run.
```

What it means

On 1,000 images the network rebuilds its training images almost perfectly (0.00161) and new ones far worse (0.01827): it memorised. With 20,000 the gap shrinks from +0.01666 to +0.00068. The learning curves show the small run's test loss turning upward after about a fifth of training: the classic sign of overfitting.

WHY CHOOSE THIS?

More data is the simplest cure for memorising.

WHY NOT THIS? (TRADEOFFS)

MNIST is generous; many real datasets are not.

6. What the code holds

DEFINITION — start here, no prior knowledge assumed

Labels are used only to judge: kNN on 5,000 reference images and k-means, on raw pixels, PCA's 32 numbers and the autoencoder's 32.

REAL-WORLD SCENARIO

Judging a summary by whether similar books end up with similar summaries.

ACTUAL OUTPUT

```
  representation            kNN accuracy (k = 3)  k-means ARI (k = 10)
  raw pixels (784)                        0.9050                0.4069
  PCA (32)                                0.9170                0.3776
  autoencoder code (32)                   0.9155                0.3875
Actual output from this lesson's run.
```

What it means

The codes are a good summary, but not a better one for this purpose: kNN 0.9155 against PCA's 0.9170, and k-means 0.3875 against 0.3776. Rebuilding pixels well is not the same as separating digits; the autoencoder was never asked to do the second.

WHY CHOOSE THIS?

Codes learned with no labels at all.

WHY NOT THIS? (TRADEOFFS)

Good reconstruction does not guarantee useful features.

7. Decoding codes it has never seen

DEFINITION — start here, no prior knowledge assumed

Codes drawn at random with the same mean and spread as real codes, decoded, and compared with the nearest of 10,000 real images.

REAL-WORLD SCENARIO

Asking the sketch artist to draw from twenty random words.

ACTUAL OUTPUT

```
distance to the nearest of 10,000 other real images (root mean square per pixel): decoded real
    codes 0.1504, decoded random codes 0.2193
random codes land in regions no training image used, and decode to blurs: the code space has gaps
    (Day 129's VAE closes them)
Actual output from this lesson's run.
```

What it means

Decoded real codes sit 0.1504 from the nearest real image; decoded random codes 0.2193, and in the figure they are blots, not digits. The training codes fill only scattered regions of the 32-dimensional space, and a random point lands in a gap the decoder never learned. Day 129's variational autoencoder is designed to close those gaps; Day 130 measures whether it does.

WHY CHOOSE THIS?

Shows why a plain autoencoder is not a generator.

WHY NOT THIS? (TRADEOFFS)

One way of drawing random codes; others could land closer to the data.

Interview preparation — technical questions

Q: What is an autoencoder?

A: A network trained to rebuild its input through a narrow code; encoder, code, decoder.

Q: How does it relate to PCA?

A: A linear one learns PCA's subspace (0.01707 against 0.01689); ReLU layers do better (0.00731).

Q: Which loss for images?

A: Cross-entropy on logits for pixels in [0, 1]; squared error for real values.

Q: How do you tell it is overfitting?

A: Training error far below test error: 0.00161 against 0.01827 on 1,000 images.

Q: Are autoencoder codes good features?

A: Not automatically: kNN 0.9155 against PCA's 0.9170 here.

Interview preparation — theoretical questions

Q: Why must the code be narrow?

A: Otherwise the network can copy the input without learning its structure.

Q: Why can a linear autoencoder not beat PCA?

A: PCA is the best linear subspace for squared error; a linear network can only find a subspace.

Q: Why do random codes decode badly?

A: The decoder only learned the regions where training codes lie (0.2193 against 0.1504).

Q: Why compute cross-entropy from logits?

A: $\log(1 + e^{l}) - x\,l$ never takes log(0), however confident the network is.

Key takeaway and what's next

> **KEY TAKEAWAY**
> An autoencoder learns to compress by rebuilding, with no labels. Linear, it is PCA by another route; with ReLU layers it compresses MNIST far better. But better reconstructions are not automatically better features, and its code space is full of gaps: two limits the rest of the week takes on.

- Linear autoencoder 0.01707 against PCA's 0.01689; subspace cosines 0.9878 to 0.9999.
- ReLU autoencoder, 32 numbers: 0.00731 against PCA's 0.01689.
- 1,000 images: training 0.00161, test 0.01827.
- Decoded random codes 0.2193 from real images; decoded real codes 0.1504.

Suggested watch order, if starting from zero

- 1. Autoencoders - Explained (DataMListic) — the encoder-code-decoder idea.
- 2. Autoencoder Explained - Deep Neural Networks (AIEngineering) — autoencoders as networks that reproduce their input.

Day 127 preview

- **Denoising autoencoders**: rebuilding clean images from damaged ones, with a real noise-removal demonstration.